

El Sistema que No Funcionó – Reflexión Individual

1. ¿Qué errores se identifican en la fase de análisis?

- **Ausencia de levantamiento formal de requisitos.** No se realizaron sesiones estructuradas con el cliente para documentar qué necesitaba exactamente el sistema. El equipo asumió funcionalidades en lugar de verificarlas.
- **Falta de participación del usuario final.** Los usuarios no fueron involucrados desde el inicio, lo que explica que el sistema no permitiera cancelar citas, una necesidad operativa básica.
- **No se definieron requisitos no funcionales.** No se establecieron criterios de rendimiento, seguridad ni usabilidad, que también forman parte del análisis.
- **Diagramas desconectados de la realidad construida.** Los modelos (casos de uso, diagramas de flujo, etc.) no reflejaban el sistema final, lo que indica que o no se actualizaron o nunca fueron validados.
- **Información duplicada en módulos.** Señal de que no hubo un modelo de datos unificado ni revisión de la arquitectura de información durante el análisis.

2. ¿Qué consecuencias tuvo no validar correctamente los requisitos?

La falta de validación desencadenó una reacción en cadena que afectó todas las dimensiones del proyecto:

Dimensión Consecuencia concreta

Funcional Funciones ausentes (cancelación de citas, reportes gerenciales)

Calidad Módulos con datos duplicados y comportamiento incorrecto

Cliente Insatisfacción: el sistema entregado no era lo que se pidió

Tiempo Retrasos por retrabajo y correcciones tardías

Costo Sobrecostos derivados de modificaciones no planificadas

Técnica Diagramas inconsistentes con el código real

El problema central es que **los errores de análisis son los más costosos de corregir**, porque se propagan a diseño, desarrollo, pruebas y entrega. Corregir un requisito mal entendido en producción puede costar hasta 100 veces más que corregirlo en la fase de análisis.

3. ¿Por qué es importante la trazabilidad en un proyecto de software?

La **trazabilidad** es la capacidad de rastrear cada elemento del sistema (módulo, función, prueba) hasta el requisito que lo originó, y viceversa.

En este caso, la ausencia de trazabilidad provocó que:

- Nadie pudiera detectar que los reportes gerenciales nunca fueron implementados hasta las pruebas finales.
- No había forma de verificar si cada requisito del cliente tenía su correspondiente diseño y desarrollo.
- Los diagramas quedaron desconectados del código sin que nadie lo notara a tiempo.

La trazabilidad es importante porque:

- **Garantiza cobertura completa:** cada requisito debe tener diseño, código y prueba asociados.
- **Facilita el control de cambios:** si un requisito cambia, se sabe exactamente qué componentes se ven afectados.
- **Permite auditoría y verificación:** el cliente y el equipo pueden confirmar que todo lo solicitado fue construido.
- **Reduce el riesgo de omisiones:** ninguna funcionalidad queda "en el aire" sin respaldo técnico.
- **Apoya el mantenimiento futuro:** un nuevo desarrollador puede entender por qué existe cada parte del sistema.

4. ¿Qué artefactos de análisis se consideran necesarios?

Para un sistema de gestión de citas médicas, los siguientes artefactos son esenciales:

Artefactos de captura de requisitos:

- **Documento de Requisitos del Software (ERS/SRS):** especificación formal de requisitos funcionales y no funcionales.

- **Historias de usuario o casos de uso:** describen qué necesita hacer cada tipo de usuario en el sistema.
- **Glosario del dominio:** define términos específicos del negocio (cita, paciente, médico, turno, etc.) para evitar ambigüedades.

Artefactos de modelado:

- **Diagrama de casos de uso UML:** muestra los actores y sus interacciones con el sistema.
- **Diagramas de flujo o actividad:** representan los procesos de negocio (agendar cita, cancelar, generar reporte).
- **Modelo Entidad-Relación (ER):** define la estructura de datos y elimina duplicaciones.
- **Prototipo o mockups de interfaz:** permiten validar visualmente con el usuario antes de desarrollar.

Artefactos de gestión y validación:

- **Matriz de Trazabilidad de Requisitos:** vincula cada requisito con su diseño, módulo y prueba.
- **Lista de criterios de aceptación:** define cuándo una funcionalidad se considera correctamente implementada.
- **Actas de reunión con el cliente:** registro firmado de acuerdos y decisiones tomadas.

5. ¿Cómo puede evitarse este tipo de problemas en futuros proyectos?

Antes de iniciar el desarrollo:

- Realizar sesiones formales de **elicitación de requisitos** con todos los interesados: cliente, usuarios finales y gerencia.
- Documentar los requisitos en un **SRS aprobado y firmado** por el cliente antes de diseñar.
- Construir **prototipos de baja fidelidad** para validar la comprensión antes de comprometer recursos de desarrollo.

Durante el análisis y diseño:

- Aplicar técnicas como entrevistas, talleres, casos de uso y prototipos para descubrir requisitos ocultos.

- Establecer desde el inicio la **Matriz de Trazabilidad** y mantenerla actualizada en cada sprint o fase.
- Revisar y validar los diagramas con el equipo técnico **y** con el cliente, no solo internamente.

Durante el desarrollo:

- Adoptar una metodología con **entregas iterativas** (Scrum, por ejemplo) para detectar desviaciones temprano.
- Realizar **revisiones de requisitos** antes de cerrar cada módulo.
- Incluir al cliente en demostraciones frecuentes para confirmar que lo construido es lo esperado.

Al cerrar el proyecto:

- Ejecutar **pruebas de aceptación del usuario (UAT)** con criterios predefinidos.
- Mantener la documentación actualizada y coherente con el sistema entregado.
- Hacer una **retrospectiva** del proyecto para identificar lecciones aprendidas.